Basics Day 11 — Session 11 (Hours 41–44)

Python Programming (Basic) • File I/O, JSON Persistence, pathlib, and Exception Handling

Session 11 Overview

Topics Covered Today

- Hour 41: Files — reading and writing text
- Hour 42: JSON persistence with the stdlib `json` module
- Hour 43: Directories and paths with `pathlib`
- Hour 44: Exception handling — full Basics treatment

Hour 41: Files — Reading and Writing Text

Learning Outcomes

- Open files safely using `with open(...) as f`
- Write lines to a text file using `write()`
- Read lines back using `readlines()` and `read()`
- Confirm file location with CWD and absolute path

Why Files Matter

Programs Forget Without Files

- Variables live only while the script runs
- Files make data **persistent** across runs

> "Memory variables are like writing on a whiteboard that gets erased when class ends. Files are like writing in a notebook you can open tomorrow."

What We Can Do with Files

- Save shopping lists, contacts, and tasks
- Store simple reports and logs
- Build apps that remember state between runs

`with open()` — The Safe Pattern

Core Syntax

```
# Write mode
with open("shopping.txt", "w", encoding="utf-8") as f:
    f.write("Apples\n")
# File is automatically closed when the block ends
```

File Modes

Mode	Meaning
`"r"`	Read (default)
`"w"`	Write — overwrites existing content
`"a"`	Append — adds to existing content

> `with` closes the file automatically, even if an error occurs.

Writing Files — write() and Newlines

write() Writes Exactly What You Pass

```
from pathlib import Path

items = ["Apples", "Bananas", "Bread", "Milk"]
file_path = Path("shopping.txt")

with open(file_path, "w", encoding="utf-8") as f:
    for item in items:
        f.write(item + "\n")   # "\n" puts each item on its own line
```

Without `\n` — All Items Merge Into One Line

```
# ❌ Missing newlines
f.write("Apples")
f.write("Bananas")
# Result in file: ApplesBananas
```

Reading Files — read() vs readlines()

readlines() — List of Lines (Use for Line-by-Line Data)

```
with open(file_path, "r", encoding="utf-8") as f:
    lines = f.readlines()   # returns ["Apples\n", "Bananas\n", ...]

for index, line in enumerate(lines, start=1):
    print(f"{index}. {line.strip()}")  # strip() removes the \n
```

read() — One Big String (Use for Full File Content)

```
with open(file_path, "r", encoding="utf-8") as f:
    entire_text = f.read()   # returns "Apples\nBananas\n..."
print(entire_text)
```

> **`strip()`** removes surrounding whitespace including `\n` — always use it when printing loaded lines.

Demo — Shopping List Save and Load

```python
from pathlib import Path

shopping_items = ["Apples", "Bananas", "Bread", "Milk"]
file_path = Path("shopping.txt")

print(f"CWD: {Path.cwd()}")
print(f"Absolute path: {file_path.resolve()}")

# Write
with open(file_path, "w", encoding="utf-8") as f:
    for item in shopping_items:
        f.write(item + "\n")
print(f"File exists? {file_path.exists()}")

# Read
with open(file_path, "r", encoding="utf-8") as f:
    lines = f.readlines()

for index, line in enumerate(lines, start=1):
    print(f"{index}. {line.strip()}")
```

Lab — Save and Load Text

Time: 25–35 minutes

Tasks

1. Create a list of tasks or contacts in Python
2. Save each item to a `.txt` file — one per line using `"w"` mode
3. Print CWD and absolute file path before writing
4. Verify the file exists after writing (`file_path.exists()`)
5. Load the lines back with `readlines()` and print them numbered

Sample Data (Tasks)

```
tasks = [
    "Review notes",
    "Email instructor",
    "Practice file I/O"
```

Common Pitfalls (Hour 41)

⚠️ **Wrong working directory** — file not found on read

→ Print `Path.cwd()` and `file_path.resolve()` to diagnose

⚠️ **Forgetting `\n`** — all items merge into one line in the file

→ Always use `f.write(item + "\n")`

⚠️ **Not using `strip()`** — extra newlines appear in printed output

→ Use `line.strip()` when printing each loaded line

Quick Check (Hour 41)

Question: Why is `with open(...)` recommended instead of calling `open()` and `close()` manually?

Strong answer: `with` automatically closes the file when the block ends — even if an error occurs inside the block. Manual `close()` can be skipped if an exception is raised, leaving the file open and risking resource leaks or data corruption.

Hour 42: JSON Persistence

Learning Outcomes

- Use `json.dump()` to save Python data to a `.json` file
- Use `json.load()` to read JSON back into Python data structures
- Identify JSON-supported types and their Python equivalents
- Convert simple class objects with `to_dict()` and `from_dict()`
- Handle `FileNotFoundError` and `json.JSONDecodeError` safely

What is JSON?

JSON — JavaScript Object Notation

- Language-neutral, human-readable text format
- Built into Python's standard library — `import json`
- Perfect for storing small app data between runs

A JSON Payload Looks Like This

```
# Equivalent Python dict written as JSON in the file:
# {
#   "name": "Ada",
#   "active": true,
#   "score": 98
# }
# Note: JSON uses lowercase true/false and null (not True/False/None)
```

> **Persistence** = writing program state to storage so it survives program restarts.

Python ↔️ JSON Type Mapping

JSON	Python on load	Python before dump
object	`dict`	`dict`
array	`list`	`list` or `tuple`
string	`str`	`str`
number	`int` or `float`	`int` or `float`
true/false	`bool`	`bool`
null	`None`	`None`

> **Important:** Tuples become lists in JSON and come back as lists after loading. Custom class objects are **not** JSON-serializable — see `to_dict` / `from_dict`.

json.dump() and json.load()

Save Python Data to a File

```
import json
from pathlib import Path

DATA_PATH = Path("data.json")

contacts = [
    {"id": 1, "name": "Ava", "email": "ava@example.com"},
    {"id": 2, "name": "Noah", "email": "noah@example.com"},
]

with DATA_PATH.open("w", encoding="utf-8") as file:
    json.dump(contacts, file, indent=2)   # indent=2 for readable output
```

Load JSON Data Back Into Python

```
with DATA_PATH.open("r", encoding="utf-8") as file:
```

Object Mapping — to_dict / from_dict

Why Custom Objects Need Conversion

```
import json

class Note:
    pass

note = Note()
# json.dump(note, file)  # ❌ TypeError: Object of type Note is not JSON serializable
```

Simple Object Mapping Pattern

```
from dataclasses import dataclass

@dataclass
class Note:
    note_id: int
    text: str
```

Handling Missing and Corrupt Files

Load with Safety Net

```
import json
from pathlib import Path

def load_data(path: Path, fallback: dict) -> dict:
    try:
        with path.open("r", encoding="utf-8") as file:
            return json.load(file)
    except FileNotFoundError:
        print(f"[INFO] {path} not found. Using fallback data.")
        return fallback
    except json.JSONDecodeError:
        print(f"[WARNING] {path} is corrupted JSON. Using fallback data.")
        return fallback
```

App Persistence Workflow

1. Program starts → try load JSON

Demo — JSON Persistence Mini App

```python
import json
from pathlib import Path

DATA_PATH = Path("data.json")
FALLBACK = {"next_id": 4, "contacts": [
    {"id": 1, "name": "Ava", "email": "ava@example.com"},
]}

def load_data(path, fallback):
    try:
        with path.open("r", encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        print("[INFO] No file found. Using defaults.")
        return fallback
    except json.JSONDecodeError:
        print("[WARNING] Corrupted JSON. Using defaults.")
        return fallback

def save_data(path, data):
    with path.open("w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)

data = load_data(DATA_PATH, FALLBACK)
save_data(DATA_PATH, data)
print("Saved:", DATA_PATH.resolve())
```

Lab — JSON Contacts App

Time: 25–35 minutes

Tasks

1. Run the `contacts_json_app.py` scaffold and list contacts
2. Add a new contact and save — re-run to confirm persistence
3. Delete `data.json` and re-run — app must NOT crash (uses fallback)
4. Manually break `data.json` syntax and re-run — app must NOT crash

Completion Criteria

✓ Data persists across multiple runs
✓ Missing file handled — no crash, fallback data used
✓ Corrupt JSON handled — no crash, fallback data used
✓ Changes saved correctly after edits

Common Pitfalls + Quick Check (Hour 42)

Common Pitfalls

⚠️ **Dumping custom object directly** — `TypeError: not JSON serializable`
→ Convert to `dict` first with `to_dict()`

⚠️ **Crashing when file missing** — no `FileNotFoundError` guard on first run
→ Wrap `json.load()` in `try/except FileNotFoundError`

⚠️ **Forgetting to save after changes** — data disappears on next run
→ Call `save_data()` before exit

⚠️ **JSON booleans** — Python `True` writes as JSON `true`; `None` writes as `null`
→ Conversion is automatic — no action needed

Quick Check

Why can't `json.dump()` write a custom object directly?

Hour 43: Directories and Paths with pathlib

Learning Outcomes

- Build reliable file paths using `pathlib.Path`
- Create a `data/` directory safely with `mkdir(exist_ok=True)`
- List directory contents with `iterdir()` and find files with `glob()`
- Choose a file deterministically from discovered results
- Explain why project-relative paths beat hardcoded personal paths

Relative vs Absolute Paths

Absolute Path — Full Location from Drive Root

```
# Windows absolute path (machine-specific — fragile!)
path = "C:/Users/Ava/Downloads/data.json"

# macOS/Linux absolute path (also machine-specific)
path = "/Users/ava/Downloads/data.json"
```

Relative Path — Relative to a Base Location

```
from pathlib import Path

# Relative to current working directory
path = Path("data/data.json")
```

> **Problem:** If you share hardcoded absolute paths, they break on every other machine.

Why CWD-Dependent Code Fails

The Common Beginner Pattern

```
# This works sometimes — fails other times
with open("data.json") as file:
    ...
```

Why It Breaks

- `"data.json"` is relative to **Current Working Directory (CWD)**
- CWD is wherever the terminal was launched — not necessarily the script's folder
- Script in `project/app.py`, terminal started in parent folder → **FileNotFoundError**

Print CWD to Diagnose (not to depend on)

```
from pathlib import Path
```

Project-Relative Base Path

Anchor Paths to the Script Location

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent   # folder containing this script
DATA_DIR = BASE_DIR / "data"                 # project/data/
```

Why This Is Reliable

- `__file__` is this script's path
- `resolve()` gives absolute, normalized location
- `.parent` gives the containing folder
- Works regardless of which folder the terminal started in

Build All Paths from BASE_DIR

Create a Folder Safely

mkdir with exist_ok=True

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
DATA_DIR = BASE_DIR / "data"

DATA_DIR.mkdir(exist_ok=True)
# exist_ok=True: create if missing, do nothing if already exists
# Scripts can be run multiple times without crashing
```

Print Location and Verify

```
print(f"Script base: {BASE_DIR}")
print(f"Data folder: {DATA_DIR}")
print(f"Folder exists? {DATA_DIR.exists()}")
```

> Always use `exist_ok=True` to make your scripts **rerunnable** without errors.

Directory Discovery — iterdir() and glob()

List Everything in a Folder

```
# All items — sorted for deterministic order
all_items = sorted(DATA_DIR.iterdir(), key=lambda p: p.name.lower())
for item in all_items:
    kind = "DIR " if item.is_dir() else "FILE"
    print(f"[{kind}] {item.name}")
```

Find Files by Pattern with glob()

```
# Only JSON files — sorted for deterministic order
json_files = sorted(DATA_DIR.glob("*.json"), key=lambda p: p.name.lower())

if not json_files:
    print("No save files found.")
else:
    chosen = json_files[0]   # deterministic: always first in sorted order
    print(f"Chosen file: {chosen}")
```

Demo — Create Folder, Save JSON, Discover Files

```python
import json
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
DATA_DIR = BASE_DIR / "data"
DATA_DIR.mkdir(exist_ok=True)

# Save a JSON file
payload = {"student": "demo_user", "score": 100}
demo_file = DATA_DIR / "save1.json"
with demo_file.open("w", encoding="utf-8") as f:
    json.dump(payload, f, indent=2)

# Discover files
json_files = sorted(DATA_DIR.glob("*.json"), key=lambda p: p.name.lower())
print("JSON files found:")
for path in json_files:
    print(f"- {path.name}")

chosen = json_files[0] if json_files else None
print(f"Chosen: {chosen}")
```

Lab — Data Folder and Deterministic Discovery

Time: 25–35 minutes

Tasks

1. Use `Path(__file__).resolve().parent` as `BASE_DIR`
2. Create `BASE_DIR / "data"` with `mkdir(exist_ok=True)`
3. Save JSON to `data/data.json`
4. Print all file names in `data/` using `iterdir()`
5. Find all `.json` files with `glob("*.json")` — sort results
6. If JSON files exist, print chosen path (first sorted); if none, print a no-files message
7. Re-run the script — confirm no crash (folder already exists)

Completion Criteria

- ✓ Paths built from script/project root — no hardcoded personal folders

Common Pitfalls + Quick Check (Hour 43)

Common Pitfalls

⚠️ **Hardcoded absolute paths** — `Path("C:/Users/Ava/Desktop/data")` breaks on all other machines
→ Use `Path(__file__).resolve().parent` as base

⚠️ **Using CWD as final path anchor** — `Path.cwd() / "data"` breaks when terminal starts elsewhere
→ Use CWD for debugging only; anchor paths to the script

⚠️ **Forgetting `exist_ok=True`** — `mkdir()` crashes if folder already exists
→ Always write `mkdir(exist_ok=True)` for rerunnable scripts

⚠️ **Assuming glob() order** — unsorted results differ across machines
→ Wrap in `sorted(... key=lambda p: p.name.lower())`

Hour 44: Exception Handling

Learning Outcomes

- Use `try/except` to handle runtime errors without crashing
- Catch specific exceptions — `ValueError`, `FileNotFoundError`, `json.JSONDecodeError`
- Apply `try/except/else/finally` for clean control flow
- Display friendly error messages and continue program execution
- Explain why specific catches are better than catch-all `except Exception`

Exceptions vs Syntax Errors

Syntax Error — Python Cannot Parse the Code

```
# ❌ Syntax error — program never starts
print("hello"   # missing closing parenthesis
```

→ Fix syntax errors in your code — `try/except` cannot catch them.

Runtime Exception — Fails During Valid Execution

```
# ✓ Valid syntax — but fails at runtime when input is wrong
user_text = input("Enter a number: ")
count = int(user_text)   # raises ValueError if input is "five"
```

→ These are what `try/except` is designed to handle.

> **Key distinction:** Syntax errors are broken keys. Runtime exceptions are blocked doors.

try/except — The Control Pattern

Minimal Pattern

```
user_text = input("Enter a whole number: ")

try:
    count = int(user_text)
except ValueError:
    print("Please enter digits only, such as 12.")
else:
    print(f"Thanks. You entered {count}.")
```

The Four-Step Mental Model

1. Put **risky code** in `try`
2. Put **recovery code** in `except` — keep the exception type specific
3. Use `else` for code that runs **only on success**

Why Specific Exceptions Matter

Specific Catch — Documents Intent, Preserves Visibility

```
try:
    value = int("abc")
except ValueError:
    print("Not a valid integer.")
# Any other exception (NameError, TypeError...) is still visible for debugging
```

Broad Catch-All — Hides Bugs

```
try:
    value = int("abc")
except Exception:
    print("Something went wrong.")
# ❌ Masks unrelated bugs — avoid in this course
```

> **Rule:** Catch expected errors. Expose unexpected errors.
>
> Never use `except Exception` everywhere, and never use bare `except:` with no

try/except/else/finally

All Four Blocks Together

```
from pathlib import Path

target = Path("data/class_settings_valid.json")

try:
    text = target.read_text(encoding="utf-8")
except FileNotFoundError:
    print(f"Could not find file: {target}")
else:
    print(f"Loaded {len(text)} characters.")   # only on success
finally:
    print("Load attempt finished.")            # always runs
```

Block Execution Summary

Scenario	`except`	`else`	`finally`

Core Expected Exceptions

ValueError — Wrong Value Shape for Conversion

```
try:
    count = int(input("Enter quantity: ").strip())
except ValueError:
    print("Friendly error: please type digits only, such as 12.")
```

FileNotFoundError — Target File Does Not Exist

```
try:
    with path.open("r", encoding="utf-8") as handle:
        settings = json.load(handle)
except FileNotFoundError:
    print("Settings file not found. Continuing with defaults.")
    settings = DEFAULT_SETTINGS.copy()
```

json.JSONDecodeError — File Exists but JSON is Invalid

Demo — Hardened CLI Tool

```python
import json
from pathlib import Path

def prompt_positive_int(prompt: str) -> int:
    while True:
        raw = input(prompt).strip()
        try:
            number = int(raw)
        except ValueError:
            print("Friendly error: please type digits only, such as 3 or 15.")
            continue
        if number <= 0:
            print("Friendly reminder: the number must be greater than zero.")
            continue
        return number

def load_settings(file_path: Path) -> dict:
    try:
        with file_path.open("r", encoding="utf-8") as handle:
            settings = json.load(handle)
    except FileNotFoundError:
        print("Settings file not found. Using defaults.")
        return {}
    except json.JSONDecodeError:
        print("Settings file has invalid JSON. Using defaults.")
        return {}
    else:
        print("Settings loaded successfully.")
        return settings
    finally:
        print("Load attempt completed.")
```

Lab — Harden Your Program

Time: 25–35 minutes

Starting Point — Two Fragile Operations

```
# Fragile: crashes on bad input or missing file
raw_count = input("How many students? ")
student_count = int(raw_count)            # crashes if "ten"

with path.open("r") as handle:
    settings = json.load(handle)          # crashes if missing or corrupt
```

Required Tasks

1. Wrap `int()` conversion in `try/except ValueError` — re-prompt on failure
2. Wrap `float()` conversion in `try/except ValueError` — re-prompt on failure
3. Wrap `json.load()` in `try/except FileNotFoundError` and

Common Pitfalls + Quick Check (Hour 44)

Common Pitfalls

⚠️ **Giant catch-all block** — `except Exception` wrapping entire program
→ Narrow your `try` block to one risky operation; catch only expected exception

⚠️ **Silent swallow with `pass`** — user sees nothing, error is hidden
→ Always print a clear, actionable message in the `except` block

⚠️ **Confusing validation with exceptions** — `if number <= 0` is not an exception
→ Use `try/except` for conversion failures; use `if` for business rule checks

⚠️ **Forgetting `return` in error branch** — function returns `None`, fails downstream
→ Every `except` branch that handles and continues must return a known fallback value

Quick Check

Session 11 Wrap-Up

What We Covered Today

Hour 41 — File I/O

- `with open()` safe context manager pattern
- `write()` with `\n`, `readlines()`, `strip()`
- Confirm file location with `Path.cwd()` and `.resolve()`

Hour 42 — JSON Persistence

- `json.dump()` to save, `json.load()` to restore
- Python ↔️ JSON type mapping
- `to_dict()` / `from_dict()` for simple class objects
- Handle `FileNotFoundError` and `JSONDecodeError`

Scope Guardrails

Stay in Basics Scope

✓ Plain text files with `with open()`
✓ JSON persistence for dicts, lists, and simple objects
✓ `pathlib` for portable, project-relative paths
✓ Specific `try/except` for expected runtime errors
✓ Friendly error messages that help users recover

Not Yet (Advanced Topics)

✗ Database drivers (SQLite, ORMs)
✗ Custom JSON encoders or serialization frameworks
✗ `logging` module (use simple `print` for now)
✗ Context managers with `__enter__` / `__exit__`
✗ Chained exceptions or exception hierarchies

What's Next — Session 12 Preview

Upcoming Hours (45–48)

- Hour 45: Lists and tuples — deeper methods and patterns
- Hour 46: Dictionaries — nested structures and common workflows
- Hour 47: Sets — membership, deduplication, and set operations
- Hour 48: Comprehensions introduction — list and dict comprehensions

Skills You Built That Carry Forward

- File I/O and JSON persistence will be used in all upcoming data exercises
- `pathlib` paths will anchor every lab's data folder
- Exception handling wraps every risky input and file operation
- These four tools together form the foundation of small, working Python apps

Thank You!

Keep Practicing

- Save your work to files — use what you learned today
- Add `try/except` to existing programs from earlier sessions
- Try breaking your own JSON files and confirm your app recovers
- Build a small contacts or tasks CLI app that persists across runs

See you in Session 12!